Read-Write Lock

Read-Write Lock · Cache manifest with many readers · Build with Waqar

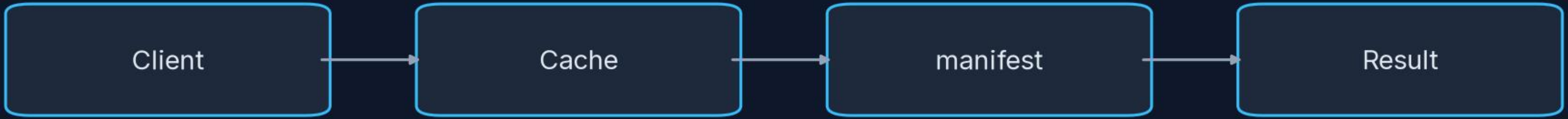
Today's Hook

- Today's force: Cache manifest with many readers
- We earn Read-Write Lock by feeling the naive pain first.
- Demo main prints a real JDK 25 trace (happy + failure).
- Key types: CachedManifest

The Problem

- Scenario: Cache manifest with many readers
- Constraints: solve «Cache manifest with many readers» with Read-Write Lock — keep the demo runnable and the failure path observable.
- Naive pain: Naive version fails to cleanly solve: Cache manifest with many readers
- Success looks like: v1

Structure


Client
Cache
manifest
Result

Read-Write Lock: Cache manifest with many readers

Real-World Scenario

- Running example: Cache manifest with many readers
- Key types: CachedManifest
- Happy-path stdout cue: v1
- Failure-path stdout cue: Failure path is asserted in ProblemTest (injected fault / invalid input)

Naive Code

```java
package com.buildwithwaqar.patterns.concurrency.readwritelock.problem;

import java.util.*;

public final class NaiveCache {
  private String v="v1"; public String read(){ return v; } public void write(String x){ v=x; }
}
```

Why Naive Hurts

- Naive version fails to cleanly solve: Cache manifest with many readers
- Without Read-Write Lock, call sites absorb the wrong details.
- Failure mode to show on camera: Failure path is asserted in ProblemTest (injected fault / invalid input)
- New requirements force shotgun edits across the naive code.

Solution

```java
package com.buildwithwaqar.patterns.concurrency.readwritelock.problem;

import java.util.*;
import java.util.concurrent.*;
import java.util.concurrent.atomic.*;
import java.util.concurrent.locks.*;
import java.util.function.*;

final class CachedManifest {
  private final ReadWriteLock lock=new ReentrantReadWriteLock();
  private String value="v1";
  String read(){ lock.readLock().lock(); try { return value; } finally { lock.readLock().unlock(); } }
  void write(String v){ lock.writeLock().lock(); try { value=v; } finally { lock.writeLock().unlock(); } }
}
```

Runnable main

```java
package com.buildwithwaqar.patterns.concurrency.readwritelock.problem;
import java.util.*; import java.util.concurrent.*; import java.util.concurrent.atomic.*;
public final class ProblemDemo {
  public static void main(String[] args) throws Exception {
    System.out.println("=== Read-Write Lock: manifest cache ===");
    System.out.println("--- naive ---"); var n=new NaiveCache(); System.out.println(n.read()); n.write("v2"); System.out.println(n.read());
System.out.println("--- rw lock ---");
var c=new CachedManifest();
ExecutorService ex=Executors.newFixedThreadPool(4);
List<Future<String>> reads=new ArrayList<>();
for(int i=0;i<10;i++) reads.add(ex.submit(c::read));
c.write("v2");
System.out.println("final="+c.read());
ex.shutdown();
  }
}
```

Walkthrough Output

```
=== Read-Write Lock: manifest cache ===
--- naive ---
v1
v2
--- rw lock ---
final=v2
```

Step-by-Step Trace

- Match stdout to stages — look for: v1
- Then the injected failure: Failure path is asserted in ProblemTest (injected fault / invalid input)
- Re-run: java -cp ... 37-read-write-lock ProblemDemo (see episode folder).
- Pause the video and predict the next printed line.

Modern Java 25 / Zazr

```
// Java 25 direction for Read-Write Lock
// Prefer records + sealed where they shrink the demo.
// Problem under test: Cache manifest with many readers
```

Trade-offs

- Use Read-Write Lock when «Cache manifest with many readers» repeats and a simpler feature does not erase it.
- Refuse extra types when a sealed switch or a function value already suffices.
- Teaching demos omit production metrics, retries, and multi-region clocks.
- Interview soundbite: state the force behind Read-Write Lock in product language.

Related Patterns

- Name the pattern usually confused with Read-Write Lock and separate the forces.
- Sketch the roles for «Cache manifest with many readers» from memory in sixty seconds.
- Point to the next episode in this season on the playlist map.
- Link this episode's series/ folder in the YouTube description.

Recap

- Problem: Cache manifest with many readers
- Naive pain → Read-Write Lock boundary → Demo stdout.
- Remember the failure cue: Failure path is asserted in ProblemTest (injected fault / invalid input)
- Break a ProblemTest assertion, watch it fail, restore it.